Campusmonk

Infosys PYQ

739. Daily Temperatures

Minimum Bit Flips to Convert Number

Campusmonk

739. Daily Temperatures

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i^th` *day to get a warmer temperature*. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

> **Input:** temperatures = [73,74,75,71,69,72,76,73]
> **Output:** [1,1,4,2,1,1,0,0]

Example 2:

> **Input:** temperatures = [30,40,50,60]
> **Output:** [1,1,1,0]

Example 3:

> **Input:** temperatures = [30,60,90]
> **Output:** [1,1,0]

Campusmonk

1. **Monotonic Increasing Stack**
   - Elements are stored in increasing order (from bottom to top).
   - Before pushing a new element, pop all larger elements.

   Example:

```
Stack: 2, 4, 7

New element = 5

Pop 7
Push 5

Result: 2, 4, 5
```

2. **Monotonic Decreasing Stack**
   - Elements are stored in decreasing order (from bottom to top).
   - Before pushing a new element, pop all smaller elements.

   Example:

```
Stack: 9, 6, 3

New element = 5

Pop 3
Push 5

Result: 9, 6, 5
```

Campusmonk

Java Auto

```java
class Solution {
    public int[] dailyTemperatures(int[] temp) {
        Stack<Integer>st = new Stack<>();
        int n=temp.length;
        int ans[]=new int[n];
        st.push(0);
        for(int i=1;i<n;i++){
        while(!st.isEmpty() && temp[i]>temp[st.peek()]){
            int ind = i-st.peek();
            ans[st.peek()]=ind;
            st.pop();
        }
        st.push(i);
        }
        return ans;
    }
}
```

Campusmonk

C++ Auto

```cpp
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temp) {
        int n = temp.size();
        vector<int> ans(n, 0);
        stack<int> st;

        st.push(0);

        for (int i = 1; i < n; i++) {
            while (!st.empty() && temp[i] > temp[st.top()]) {
                int ind = i - st.top();
                ans[st.top()] = ind;
                st.pop();
            }
            st.push(i);
        }

        return ans;
    }
};
```

Campusmonk

Python3 Auto

```python
class Solution:
    def dailyTemperatures(self, temp: List[int]) -> List[i
        n = len(temp)
        ans = [0] * n
        st = [0]

        for i in range(1, n):
            while st and temp[i] > temp[st[-1]]:
                ind = i - st[-1]
                ans[st[-1]] = ind
                st.pop()
            st.append(i)

        return ans
```

Campusmonk

Minimum Bit Flips to Convert Number

Given two integers start and goal. Flip the **minimum** number of bits of start integer to convert it into goal integer.

A bits flip in the number val is to choose any bit in binary representation of val and flipping it from either 0 to 1 or 1 to 0.

Example 1

> **Input :** start = 10 , goal = 7
> **Output :** 3
> **Explanation :** The binary representation of 10 is "1010".
> The binary representation of 7 is "111".
> If we flip the underlined bits in binary representation of 10 then we will obtain our goal.

Example 2

> **Input :** start = 3 , goal = 4
> **Output :** 3
> **Explanation :** The binary representation of 3 is "011".
> The binary representation of 4 is "100".
> So if we flip all the three bits of 3 then we will reach our goal number.

Campusmonk

```java
/* Function to get the minimum
 bit flips to convert number */
public int minBitsFlip(int start, int goal) {

    /* Variable to store bits that
    are different in both numbers */
    int num = start ^ goal;

    /* Variable to count
    number of set bits */
    int count = 0;

    for (int i = 0; i < 32; i++) {
        /* Update count if the
        rightmost bit is set */
        count += (num & 1);

        /* Shift the number every
        time by 1 place */
        num = num >> 1;
    }
    return count;
}
```

```java
/* Function to get the minimum
 bit flips to convert number */
int minBitsFlip(int start, int goal) {

    /* Variable to store bits that
    are different in both numbers */
    int num = start ^ goal;

    /* Variable to count
    number of set bits */
    int count = 0;

    for(int i = 0; i < 32; i++) {
        /* Update count if the
        rightmost bit is set */
        count += (num & 1);

        /* Shift the number every
        time by 1 place */
        num = num >> 1;
    }
    return count;
}
```

Campusmonk

```python
# Function to get the minimum
# bit flips to convert number
def minBitsFlip(self, start, goal):

    # Variable to store bits that
    # are different in both numbers
    num = start ^ goal

    # Variable to count
    # number of set bits
    count = 0

    for i in range(32):
        # Update count if the
        # rightmost bit is set
        count += (num & 1)

        # Shift the number every
        # time by 1 place
        num = num >> 1

    return count
```